EXAMEN RDA-1 DE PROGRAMACIÓN MÓVIL

TÍTULO DEL EXAMEN: SafePass 2026: Sistema de Gestión de Check-in y Validación de Asistentes

OBJETIVOS DEL EXAMEN:

1. **Modelado y Seguridad de Datos:** Evaluar la capacidad de implementar modelos inmutables mediante `data classes` y gestionar estados complejos de la interfaz de usuario utilizando `sealed classes`.
2. **Programación Funcional e Idiomática:** Demostrar el dominio de `Scope Functions` (`let`, `apply`, `run`), `Extension Functions` y `Lambdas` para procesar información de forma concisa y segura.
3. **Arquitectura y Blindaje de Nulos:** Aplicar el sistema de tipos de Kotlin y operadores de seguridad (`?.`, `?:`) para garantizar una aplicación robusta que maneje entradas de usuario impredecibles sin cierres forzados (crashes).

CONTEXTO:

En el marco del evento tecnológico "TechEvent 2026", se requiere una aplicación móvil para el personal de registro. La aplicación debe permitir ingresar los datos de un asistente (Nombre, Edad y Tipo de Entrada), procesar esta información para validar si el asistente cumple con los requisitos de seguridad del evento y mostrar un resumen del estado del registro. La estabilidad es crítica, por lo que cualquier dato faltante o incorrecto debe ser gestionado mediante los contratos de seguridad de Kotlin estudiados en las primeras tres semanas.

ARQUITECTURA GENERAL DE LA APLICACIÓN:

La solución debe seguir la estructura técnica trabajada en los laboratorios previos:

- **Modelo de Datos:** Un `data class` llamado `Asistente` que contenga propiedades inmutables (`val`) para el nombre, edad (anulable) y tipo de entrada.
- **Gestión de Estados (UI State):** Una `sealed class` llamada `RegistroState` con al menos tres estados: `Idle` (esperando datos), `Success` (datos válidos y procesados) y `Error` (datos inválidos o faltantes).
- **Lógica de Negocio:**
  - Uso de una **Extension Function** sobre el tipo `String` o `Int` para validar reglas (ej. `Int.esMayorDeEdad()`).
  - Implementación de una **Higher-Order Function** que reciba una lambda para aplicar un "descuento de reserva" o "validación de prioridad" al procesar el asistente.
  - Uso obligatorio de **Scope Functions**: `apply` para configurar el objeto `Asistente` y `let` para realizar operaciones solo si los valores de entrada no son nulos.
- **Interfaz de Usuario (Compose):**
  - Un Composable principal que utilice `Scaffold` y `Column`.
  - Uso de `when` como expresión exhaustiva para renderizar la UI dependiendo del estado de `RegistroState`.

- Manejo de entradas mediante campos de texto que utilicen `toDoubleOrNull()` o `toIntOrNull()` para prevenir errores de formato.

RESULTADOS ESPERADOS:

- **Entrada Segura:** Al ingresar texto en el campo de "Edad" o dejar campos vacíos, la aplicación no debe cerrarse. Debe mostrar un mensaje de error elegante o usar valores por defecto mediante el operador Elvis (`?:`).
- **Procesamiento Reactivo:** Al presionar un botón "Registrar", la aplicación debe transformar el estado a `Success` si los datos son correctos, mostrando un resumen del asistente (usando plantillas de cadena `$`).
- **Validación de Reglas:** Si el asistente es menor de edad o el nombre está vacío, el sistema debe transicionar al estado de `Error` con una descripción del problema.
- **Estética y Estándar:** La UI debe ocupar todo el espacio disponible (Edge-to-Edge) siguiendo la configuración de la API 36 (Android 16) establecida en el Laboratorio 1.

ENTREGABLES:

1. **Proyecto en Android Studio:** Carpeta completa del proyecto (comprimida en .zip) bajo la estandarización técnica: Java 21, API 36 y Gradle blindado.
2. **Control de Versiones:** Historial de al menos 4 commits en un repositorio público de GitHub, reflejando el progreso (ej: "feat: data model", "feat: logic validation", "feat: ui state").
3. **Documento PDF de Informe:** Informe con nivel académico con al menos los siguientes elementos:

    - Portada con integrantes.
    - Introducción y contexto.
    - Captura de pantalla de la estructura de paquetes y archivos.
    - Capturas del emulador (API 36) mostrando tres escenarios: (1) Pantalla inicial, (2) Registro exitoso con datos válidos, (3) Manejo de error con datos inválidos o nulos.
    - Enlace (URL) público al repositorio de GitHub.
    - Breve explicación (máximo 150 palabras) de cómo se utilizó una `sealed class` para mejorar la seguridad de la interfaz.
    - Conclusiones y recomendaciones
    - Bibliografía.
    - Copia de todo el código de la aplicación como Anexo.
    - Cumplimiento de las normas APA.

RÚBRICA DE EVALUACIÓN

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.